

Docker

Docker. Что это такое?

- Docker - система контейнеризации (создание, запуск контейнера и управление им).
- А что такое контейнер и зачем он нужен?



Предтече

- Когда-то (даже сейчас) приложения запускались на отдельных виртуальных машинах.
- Эти виртуальные машины нужно настраивать.
- Для Production и Dev окружения машины могли быть разными (да даже в рамках окружения такое могло быть).
- Чтобы добавить ресурсов, нужно остановить машину. Чтобы добавить реплик, нужно создать ещё одну машину.
- Это приводит к труднодиагностируемым багам и большому неудобству в эксплуатации.

Контейнер

- Контейнер - приложение вместе со всем необходимым окружением - минимальная ОС, рантайм, библиотеки, веб-сервер и т.д.
- Это хорошо тем, что собранный контейнер пригоден для запуска в разных окружениях и не зависит от них (будь то разные ОС, разные машины и т.д.)
- Существует несколько систем контейнеризации, почему Docker?

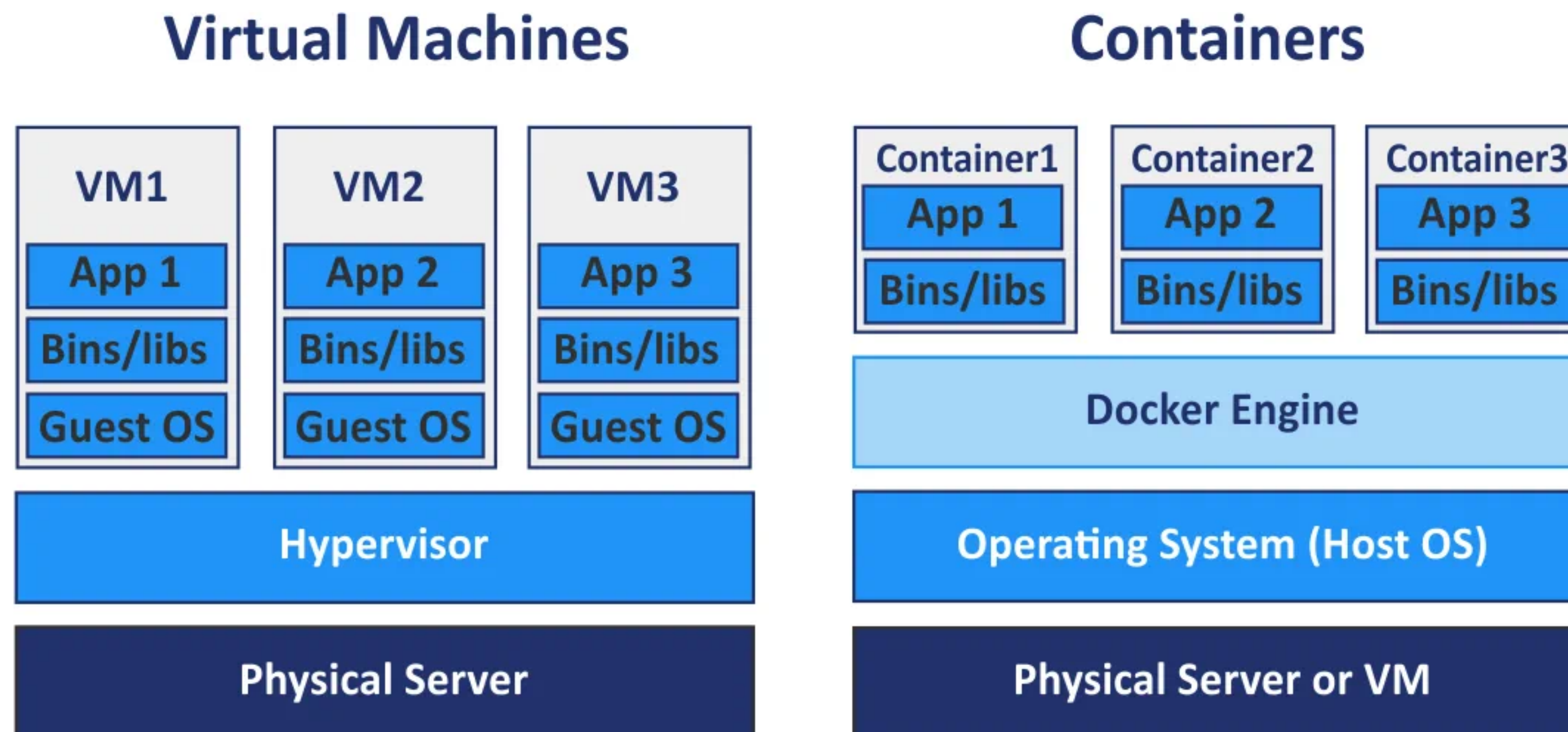
- На самом деле, вопрос стоит неправильно. Часто используется не сам Docker, а именно Docker-контейнеры. Они выбраны потому, что существует огромное количество инструментов для них, они хорошо переносимы (в отличие от HyperV/Windows-контейнеров).
- Что же касается самого Docker, то у него есть альтернативы, которые довольно часто используются (Podman, containerd, который часто используется в Kubernetes, etc)



Архитектура

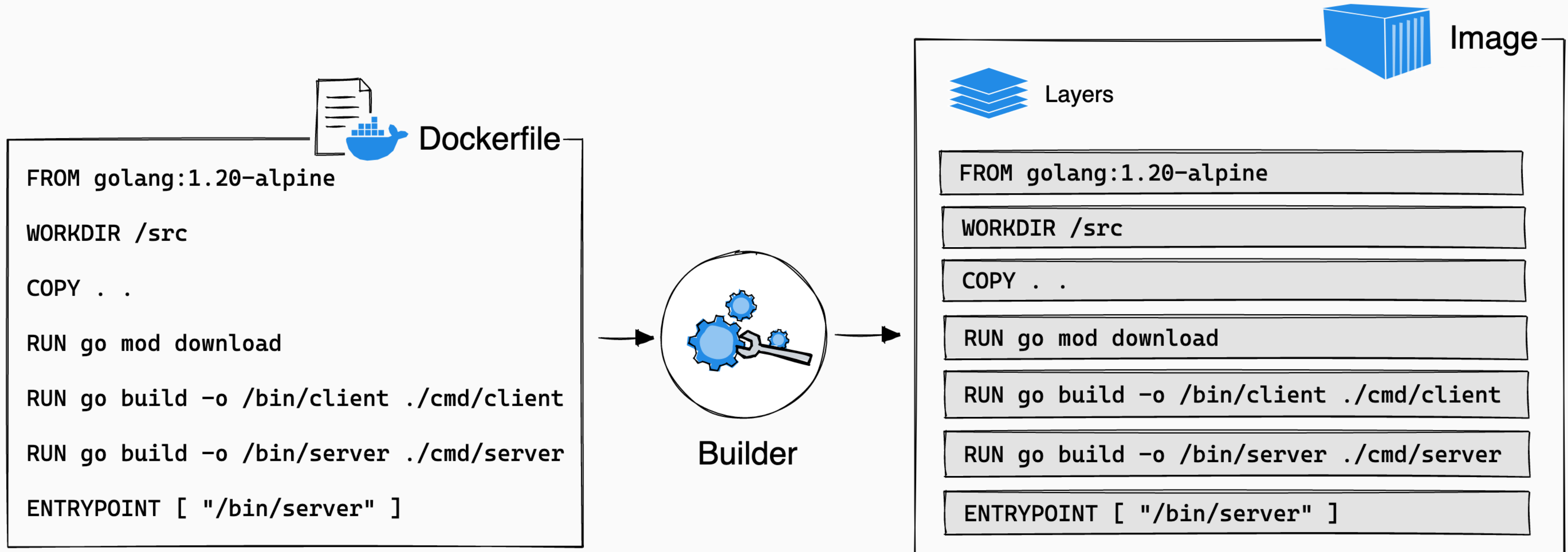
Чем контейнеризация отличается от виртуальных машин?

- Контейнер работает поверх родительской ОС, это не полноценная виртуальная машина. Он работает за счёт изоляции ресурсов посредством механизма cgroups в Linux.



Docker-образ

- Docker-образ - то, из чего будет создан контейнер. Это собранные вместе минимальная ОС, рантайм, приложение.
- Docker-образ описывается с помощью Dockerfile - файл с командами по сборке образа.
- Docker-образ состоит из слоёв. Слой - команда, результат выполнения которой можно переиспользовать.
- Не всякая команда может создавать слой. Таких четыре: FROM, RUN, COPY, ADD.
- Всего команд для сборки образа 18.



- У образа может быть базовый образ, с которого начинается сборка (задаётся командой FROM).
- Это удобно тем, что в базовых образах уже есть вся инфраструктура для сборки и запуска приложения (например, SDK и рантайм).
- Базовых образов может быть несколько.
- Примеры базовых образов - `dotnet/aspnet:8.0` и `dotnet/sdk:8.0`.

Docker registry

- Registry - хранилище Docker-образов. Может быть как облачным, так и self-hosted (обычно на базе Sonatype Nexus).
- Самый известный registry - Docker Hub.

Работа с Docker

- Обычно разработчика интересует несколько вещей:
 - сборка образа;
 - запуск контейнера;
 - запуск нескольких контейнеров;
 - что-нибудь поделаться внутри контейнера (посмотреть ФС, проверить сеть).
- Способ запуска нескольких контейнеров мы рассматривать здесь не будем - это отдельная большая тема.

Сборка образа

- `docker build <flags> <path>`
path - путь до директории, где лежит Dockerfile
- Популярные флаги:
 - t: указать тег (для нескольких тегов флаг дублируется);
 - тег - метка на образе, которая позволяет обращаться использовать эту метку в качестве имени образа, а не sha256-хэш;
 - обычно тег включает в себя версию образа (после двоеточия), если версии нет, проставляется «latest»
 - f: выбор Dockerfile для сборки, если их несколько.

Запуск контейнера

- `docker run <options> <image> <command> <args>`
- - `options` - опции для `docker run`;
 - `image` - sha256-хэш или тег образа;
 - `command` - команда для запуска при старте контейнера (об этом дальше);
 - `args` - аргументы для команды из `<command>`

docker run

<options>

- Позволяют настроить ресурсы, изоляцию, переопределить какие-то параметры внутри образа (типа `entrypoint`).
- Помимо них, существует несколько популярных опций:
 - v (`--volume`): монтирование директорий/файлов/других видов `volume` в контейнер;
 - it - запуск команды при старте;
 - p - маппинг портов (для нескольких опция повторяется). Нужно для веб-сервисов.

Volume

- Персистентное хранилище, которое можно подключить к контейнеру.
- Может быть файлом, директорией, диском, чем-то удалённым, доступным по сети.
- Можно создавать на постоянной основе, можно временно подключать при старте конкретного контейнера:
-v путь_на_хосте:/абсолютный/путь/в/контейнере
- На хосте можно относительные и абсолютные пути, в контейнере - абсолютные.

docker run

<command> + <args>

- Это должна быть команда внутри контейнера. Часто запускают какой-нибудь интерпретатор команд, чтобы можно было посмотреть всё, что интересует.
Пример:
`docker run -it dotnet/sdk:10 /bin/bash`
- Параметры для команды при необходимости можно передать сразу же после названия команды (через пробел).

docker run

-p

- Позволяет выставить из контейнера наружу порты.
-p src_port:dst_port
src_port - порт в контейнере, dst_port - на хосте
- Можно выставлять несколько повтором опций.
- Если на хосте порт занят - контейнер не запустится.

docker attach

- `docker attach <container_id>`
- Позволяет подключиться к уже работающему контейнеру.